Chapter 3: pytest Fixtures

Managing Test Data, State Setup, and Mocking External Systems

Source: Python Testing with pytest (Brian Okken) | Study Edition for Autonomous Calendar Scheduler (Week 1)

Table of Contents & Core Learning Outline

• **Why Use Fixtures?** — Separating preparation and cleanup code from actual test assertions

• **Sharing Fixtures via conftest.py** — Making test datasets globally available without manual imports

• **Fixture Setup & Teardown** — Using yield statements for deterministic cleanup

• **Fixture Scopes** — Controlling execution lifecycle: function, class, module, and session scope

• **Fixture Composition** — Building layered fixtures that depend on other fixtures

• **Parametrizing Fixtures** — Executing multiple test runs with varying mock configurations

---

Print & Study Note: *This document contains the full unabridged chapter text following this Table of Contents page. Focus on understanding the core design patterns and self-implementing the exercises without copy-pasting AI code.*

Chapter 3
pytest Fixtures

Now that you've seen the basics of pytest, let's turn our attention to fixtures, which are essential to structuring test code for almost any non-trivial software system. Fixtures are functions that are run by pytest before (and sometimes after) the actual test functions. The code in the fixture can do whatever you want it to. You can use fixtures to get a data set for the tests to work on. You can use fixtures to get a system into a known state before running a test. Fixtures are also used to get data ready for multiple tests.

Here's a simple fixture that returns a number:

ch3/test_fixtures.py

```
import pytest


@pytest.fixture()
def some_data():
"""Return answer to ultimate question."""
return 42


def test_some_data(some_data):
"""Use fixture return value in a test."""
assert some_data == 42
```

The @pytest.fixture() decorator is used to tell pytest that a function is a fixture. When you include the fixture name in the parameter list of a test function, pytest knows to run it before running the test. Fixtures can do work, and can also return data to the test function.

The test test_some_data() has the name of the fixture, some_data, as a parameter. pytest will see this and look for a fixture with this name. Naming is significant in pytest. pytest will look in the module of the test for a fixture of that name. It will also look in conftest.py files if it doesn't find it in this file.

Before we start our exploration of fixtures (and the conftest.py file), I need to address the fact that the term fixture has many meanings in the programming and test community, and even in the Python community. I use "fixture," "fixture function," and "fixture method" interchangeably to refer to the @pytest.fixture() decorated functions discussed in this chapter. Fixture can also be used to refer to the resource that is being set up by the fixture functions. Fixture functions often set up or retrieve some data that the test can work with. Sometimes this data is considered a fixture. For example, the Django community often uses fixture to mean some initial data that gets loaded into a database at the start of an application.

Regardless of other meanings, in pytest and in this book, test fixtures refer to the mechanism

pytest provides to allow the separation of “getting ready for” and “cleaning up after” code from your test functions.

pytest fixtures are one of the unique core features that make pytest stand out above other test frameworks, and are the reason why many people switch to and stay with pytest. However, fixtures in pytest are different than fixtures in Django and different than the setup and teardown procedures found in unittest and nose. There are a lot of features and nuances about fixtures. Once you get a good mental model of how they work, they will seem easy to you. However, you have to play with them a while to get there, so let’s get started.

Sharing Fixtures Through conftest.py

You can put fixtures into individual test files, but to share fixtures among multiple test files, you need to use a conftest.py file somewhere centrally located for all of the tests. For the Tasks project, all of the fixtures will be in tasks_proj/tests/conftest.py.

From there, the fixtures can be shared by any test. You can put fixtures in individual test files if you want the fixture to only be used by tests in that file. Likewise, you can have other conftest.py files in subdirectories of the top tests directory. If you do, fixtures defined in these lower-level conftest.py files will be available to tests in that directory and subdirectories. So far, however, the fixtures in the Tasks project are intended to be available to any test. Therefore, putting all of our fixtures in the conftest.py file at the test root, tasks_proj/tests, makes the most sense.

Although conftest.py is a Python module, it should not be imported by test files. Don't import conftest from anywhere. The conftest.py file gets read by pytest, and is considered a local plugin, which will make sense once we start talking about plugins in Chapter 5, *Plugins*. For now, think of tests/conftest.py as a place where we can put fixtures used by all tests under the tests directory.

Next, let's rework some our tests for tasks_proj to properly use fixtures.

Using Fixtures for Setup and Teardown

Most of the tests in the Tasks project will assume that the Tasks database is already set up and running and ready. And we should clean things up at the end if there is any cleanup needed. And maybe also disconnect from the database. Luckily, most of this is taken care of within the tasks code with tasks.start_tasks_db(<directory to store db>, 'tiny' or 'mongo') and tasks.stop_tasks_db(); we just need to call them at the right time, and we need a temporary directory.

Fortunately, pytest includes a cool fixture called tmpdir that we can use for testing and don't have to worry about cleaning up. It's not magic, just good coding by the pytest folks. (Don't worry; we look at tmpdir and it's session-scoped relative tmpdir_factory in more depth in *Using tmpdir and tmpdir_factory*.)

Given those pieces, this fixture works nicely:

ch3/a/tasks_proj/tests/conftest.py

```
import pytest
import tasks
from tasks import Task


@pytest.fixture()
def tasks_db(tmpdir):
"""Connect to db before tests, disconnect after."""
# Setup : start db
tasks.start_tasks_db(str(tmpdir), 'tiny')

yield # this is where the testing happens

# Teardown : stop db
tasks.stop_tasks_db()
```

The value of tmpdir isn't a string—it's an object that represents a directory. However, it implements __str__, so we can use str() to get a string to pass to start_tasks_db(). We're still using 'tiny' for TinyDB, for now.

A fixture function runs before the tests that use it. However, if there is a yield in the function, it stops there, passes control to the tests, and picks up on the next line after the tests are done. Therefore, think of the code above the yield as "setup" and the code after yield as "teardown." The code after the yield, the "teardown," is guaranteed to run regardless of what happens during the tests. We're not returning any data with the yield in this fixture. But you can.

Let's change one of our tasks.add() tests to use this fixture:

ch3/a/tasks_proj/tests/func/test_add.py

```
import pytest
import tasks
from tasks import Task


def test_add_returns_valid_id(tasks_db):
    """tasks.add(<valid task>) should return an integer."""
    # GIVEN an initialized tasks db
    # WHEN a new task is added
    # THEN returned task_id is of type int
    new_task = Task('do something')
    task_id = tasks.add(new_task)
    assert isinstance(task_id, int)
```

The main change here is that the extra fixture in the file has been removed, and we've added tasks_db to the parameter list of the test. I like to structure tests in a GIVEN/WHEN/THEN format using comments, especially when it isn't obvious from the code what's going on. I think it's helpful in this case. Hopefully, GIVEN an initialized tasks db helps to clarify why tasks_db is used as a fixture for the test.

Make Sure Tasks Is Installed



We're still writing tests to be run against the Tasks project in this chapter, which was first installed in Chapter 2. If you skipped that chapter, be sure to install tasks with cd code; pip install ./tasks_proj/.

Tracing Fixture Execution with –setup-show

If you run the test from the last section, you don't get to see what fixtures are run:

```
$ cd /path/to/code/
$ pip install ./tasks_proj/ # if not installed yet
$ cd /path/to/code/ch3/a/tasks_proj/tests/func
$ pytest -v test_add.py -k valid_id
==================== test session starts ====================
collected 3 items

test_add.py::test_add_returns_valid_id PASSED

==================== 2 tests deselected ====================
============ 1 passed, 2 deselected in 0.02 seconds ============
```

When I'm developing fixtures, I like to see what's running and when. Fortunately, pytest provides a command-line flag, --setup-show, that does just that:

```
$ pytest --setup-show test_add.py -k valid_id
==================== test session starts ====================
collected 3 items

test_add.py
SETUP S tmpdir_factory
SETUP F tmpdir (fixtures used: tmpdir_factory)
SETUP F tasks_db (fixtures used: tmpdir)
func/test_add.py::test_add_returns_valid_id
(fixtures used: tasks_db, tmpdir, tmpdir_factory).
TEARDOWN F tasks_db
TEARDOWN F tmpdir
TEARDOWN S tmpdir_factory

==================== 2 tests deselected ====================
============ 1 passed, 2 deselected in 0.02 seconds ============
```

Our test is in the middle, and pytest designates a SETUP and TEARDOWN portion to each fixture. Going from test_add_returns_valid_id up, you see that tmpdir ran before the test. And before that, tmpdir_factory. Apparently, tmpdir uses it as a fixture.

The F and S in front of the fixture names indicate scope. F for function scope, and S for session scope. I'll talk about scope in *Specifying Fixture Scope*.

Using Fixtures for Test Data

Fixtures are a great place to store data to use for testing. You can return anything. Here's a fixture returning a tuple of mixed type:

ch3/test_fixtures.py

```
@pytest.fixture()
def a_tuple():
"""Return something more interesting."""
return (1, 'foo', None, {'bar': 23})


def test_a_tuple(a_tuple):
"""Demo the a_tuple fixture."""
assert a_tuple[3]['bar'] == 32
```

Since test_a_tuple() should fail (23 != 32), we can see what happens when a test with a fixture fails:

```
$ cd /path/to/code/ch3
$ pytest test_fixtures.py::test_a_tuple
====================== test session starts ======================
collected 1 item

test_fixtures.py F


============================ FAILURES ===========================
_____________________________ test_a_tuple _____________________________

a_tuple = (1, 'foo', None, {'bar': 23})

def test_a_tuple(a_tuple):
"""Demo the a_tuple fixture."""
> assert a_tuple[3]['bar'] == 32
E assert 23 == 32

test_fixtures.py:43: AssertionError
=================== 1 failed in 0.07 seconds ====================
```

Along with the stack trace section, pytest reports the value parameters of the function that raised the exception or failed an assert. In the case of tests, the fixtures are parameters to the test, and

are therefore reported with the stack trace.

What happens if the assert (or any exception) happens in the fixture?

```
$ pytest -v test_fixtures.py::test_other_data
==================== test session starts ====================
collected 1 item

test_fixtures.py::test_other_data ERROR

============================ ERRORS ============================
_______________ ERROR at setup of test_other_data _______________

@pytest.fixture()
def some_other_data():
"""Raise an exception from fixture."""
x = 43
> assert x == 42
E assert 43 == 42

test_fixtures.py:24: AssertionError
=================== 1 error in 0.04 seconds ====================
```

A couple of things happen. The stack trace shows correctly that the assert happened in the fixture function. Also, test_other_data is reported not as FAIL, but as ERROR. This distinction is great. If a test ever fails, you know the failure happened in the test proper, and not in any fixture it depends on.

But what about the Tasks project? For the Tasks project, we could probably use some data fixtures, perhaps different lists of tasks with various properties:

ch3/a/tasks_proj/tests/conftest.py

```
# Reminder of Task constructor interface
# Task(summary=None, owner=None, done=False, id=None)
# summary is required
# owner and done are optional
# id is set by database

@pytest.fixture()
def tasks_just_a_few():
"""All summaries and owners are unique."""
return (
Task('Write some code', 'Brian', True),
Task("Code review Brian's code", 'Katie', False),
```

```
    Task('Fix what Brian did', 'Michelle', False))


@pytest.fixture()
def tasks_mult_per_owner():
"""Several owners with several tasks each."""
return (
Task('Make a cookie', 'Raphael'),
Task('Use an emoji', 'Raphael'),
Task('Move to Berlin', 'Raphael'),

Task('Create', 'Michelle'),
Task('Inspire', 'Michelle'),
Task('Encourage', 'Michelle'),

Task('Do a handstand', 'Daniel'),
Task('Write some books', 'Daniel'),
Task('Eat ice cream', 'Daniel'))
```

You can use these directly from tests, or you can use them from other fixtures. Let's use them to build up some non-empty databases to use for testing.

Using Multiple Fixtures

You've already seen that tmpdir uses tmpdir_factory. And you used tmpdir in our tasks_db fixture. Let's keep the chain going and add some specialized fixtures for non-empty tasks databases:

ch3/a/tasks_proj/tests/conftest.py

```
@pytest.fixture()
def db_with_3_tasks(tasks_db, tasks_just_a_few):
"""Connected db with 3 tasks, all unique."""
for t in tasks_just_a_few:
tasks.add(t)


@pytest.fixture()
def db_with_multi_per_owner(tasks_db, tasks_mult_per_owner):
"""Connected db with 9 tasks, 3 owners, all with 3 tasks."""
for t in tasks_mult_per_owner:
tasks.add(t)
```

These fixtures all include two fixtures each in their parameter list: tasks_db and a data set. The data set is used to add tasks to the database. Now tests can use these when you want the test to start from a non-empty database, like this:

ch3/a/tasks_proj/tests/func/test_add.py

```
def test_add_increases_count(db_with_3_tasks):
"""Test tasks.add() affect on tasks.count()."""
# GIVEN a db with 3 tasks
# WHEN another task is added
tasks.add(Task('throw a party'))

# THEN the count increases by 1
assert tasks.count() == 4
```

This also demonstrates one of the great reasons to use fixtures: to focus the test on what you're actually testing, not on what you had to do to get ready for the test. I like using comments for GIVEN/WHEN/THEN and trying to push as much GIVEN into fixtures for two reasons. First, it makes the test more readable and, therefore, more maintainable. Second, an assert or exception in the fixture results in an ERROR, while an assert or exception in a test function results in a FAIL. I don't want test_add_increases_count() to FAIL if database initialization failed. That would just be confusing. I want a FAIL for test_add_increases_count() to only be possible if add() really failed to alter the count. Let's trace it and see all the fixtures run:

```
$ cd /path/to/code/ch3/a/tasks_proj/tests/func
```

```
$ pytest --setup-show test_add.py::test_add_increases_count
========== test session starts ==========
collected 1 item

test_add.py
SETUP S tmpdir_factory
SETUP F tmpdir (fixtures used: tmpdir_factory)
SETUP F tasks_db (fixtures used: tmpdir)
SETUP F tasks_just_a_few
SETUP F db_with_3_tasks (fixtures used: tasks_db, tasks_just_a_few)
func/test_add.py::test_add_increases_count
(fixtures used: db_with_3_tasks, tasks_db, tasks_just_a_few,
tmpdir, tmpdir_factory).
TEARDOWN F db_with_3_tasks
TEARDOWN F tasks_just_a_few
TEARDOWN F tasks_db
TEARDOWN F tmpdir
TEARDOWN S tmpdir_factory

========== 1 passed in 0.04 seconds ==========
```

There are those F's and S's for function and session scope again. Let's learn about those next.

Specifying Fixture Scope

Fixtures include an optional parameter called scope, which controls how often a fixture gets set up and torn down. The scope parameter to @pytest.fixture() can have the values of function, class, module, or session. The default scope is function. The tasks_db fixture and all of the fixtures so far don't specify a scope. Therefore, they are function scope fixtures.

Here's a rundown of each scope value:

scope='function'

> Run once per test function. The setup portion is run before each test using the fixture. The teardown portion is run after each test using the fixture. This is the default scope used when no scope parameter is specified.

scope='class'

> Run once per test class, regardless of how many test methods are in the class.

scope='module'

> Run once per module, regardless of how many test functions or methods or other fixtures in the module use it.

scope='session'

> Run once per session. All test methods and functions using a fixture of session scope share one setup and teardown call.

Here's how the scope values look in action:

ch3/test_scope.py

```
"""Demo fixture scope."""

import pytest


@pytest.fixture(scope='function')
def func_scope():
"""A function scope fixture."""


@pytest.fixture(scope='module')
def mod_scope():
"""A module scope fixture."""
```

```
@pytest.fixture(scope='session')
def sess_scope():
"""A session scope fixture."""


@pytest.fixture(scope='class')
def class_scope():
"""A class scope fixture."""


def test_1(sess_scope, mod_scope, func_scope):
"""Test using session, module, and function scope fixtures."""


def test_2(sess_scope, mod_scope, func_scope):
"""Demo is more fun with multiple tests."""

@pytest.mark.usefixtures('class_scope')
class TestSomething():
"""Demo class scope fixtures."""

def test_3(self):
"""Test using a class scope fixture."""

def test_4(self):
"""Again, multiple tests are more fun."""
```

Let's use --setup-show to demonstrate that the number of times a fixture is called and when the setup and teardown are run depend on the scope:

```
$ cd /path/to/code/ch3
$ pytest --setup-show test_scope.py
======================== test session starts ========================
collected 4 items

test_scope.py
SETUP S sess_scope
SETUP M mod_scope
SETUP F func_scope
test_scope.py::test_1
(fixtures used: func_scope, mod_scope, sess_scope).
TEARDOWN F func_scope
```

```
SETUP F func_scope
test_scope.py::test_2
(fixtures used: func_scope, mod_scope, sess_scope).
TEARDOWN F func_scope
SETUP C class_scope
test_scope.py::TestSomething::()::test_3 (fixtures used: class_scope).
test_scope.py::TestSomething::()::test_4 (fixtures used: class_scope).
TEARDOWN C class_scope
TEARDOWN M mod_scope
TEARDOWN S sess_scope

==================== 4 passed in 0.01 seconds ====================
```

Now you get to see not just F and S for function and session, but also C and M for class and module.

Scope is defined with the fixture. I know this is obvious from the code, but it's an important point to make sure you fully grok. The scope is set at the definition of a fixture, and not at the place where it's called. The test functions that use a fixture don't control how often a fixture is set up and torn down.

Fixtures can only depend on other fixtures of their same scope or wider. So a function scope fixture can depend on other function scope fixtures (the default, and used in the Tasks project so far). A function scope fixture can also depend on class, module, and session scope fixtures, but you can't go in the reverse order.

Changing Scope for Tasks Project Fixtures

With this knowledge of scope, let's now change the scope of some of the Task project fixtures.

So far, we haven't had a problem with test times. But it seems like a waste to set up a temporary directory and new connection to a database for every test. As long as we can ensure an empty database when needed, that should be sufficient.

To have something like tasks_db be session scope, you need to use tmpdir_factory, since tmpdir is function scope and tmpdir_factory is session scope. Luckily, this is just a one-line code change (well, two if you count tmpdir -> tmpdir_factory in the parameter list):

ch3/b/tasks_proj/tests/conftest.py

```
import pytest
import tasks
from tasks import Task


@pytest.fixture(scope='session')
def tasks_db_session(tmpdir_factory):
"""Connect to db before tests, disconnect after."""
```

```
    temp_dir = tmpdir_factory.mktemp('temp')
    tasks.start_tasks_db(str(temp_dir), 'tiny')
    yield
    tasks.stop_tasks_db()


@pytest.fixture()
def tasks_db(tasks_db_session):
    """An empty tasks db."""
    tasks.delete_all()
```

Here we changed tasks_db to depend on tasks_db_session, and we deleted all the entries to make sure it's empty. Because we didn't change its name, none of the fixtures or tests that already include it have to change.

The data fixtures just return a value, so there really is no reason to have them run all the time. Once per session is sufficient:

ch3/b/tasks_proj/tests/conftest.py

```
# Reminder of Task constructor interface
# Task(summary=None, owner=None, done=False, id=None)
# summary is required
# owner and done are optional
# id is set by database
@pytest.fixture(scope='session')
def tasks_just_a_few():
    """All summaries and owners are unique."""
    return (
        Task('Write some code', 'Brian', True),
        Task("Code review Brian's code", 'Katie', False),
        Task('Fix what Brian did', 'Michelle', False))


@pytest.fixture(scope='session')
def tasks_mult_per_owner():
    """Several owners with several tasks each."""
    return (
        Task('Make a cookie', 'Raphael'),
        Task('Use an emoji', 'Raphael'),
        Task('Move to Berlin', 'Raphael'),

        Task('Create', 'Michelle'),
        Task('Inspire', 'Michelle'),
```

```
    Task('Encourage', 'Michelle'),

    Task('Do a handstand', 'Daniel'),
    Task('Write some books', 'Daniel'),
    Task('Eat ice cream', 'Daniel'))
```

Now, let's see if all of these changes work with our tests:

```
$ cd /path/to/code/ch3/b/tasks_proj
$ pytest
==================== test session starts ====================
collected 55 items

tests/func/test_add.py ...
tests/func/test_add_variety.py ............................
tests/func/test_add_variety2.py ............
tests/func/test_api_exceptions.py .......
tests/func/test_unique_id.py .
tests/unit/test_task.py ....

=================== 55 passed in 0.17 seconds ===================
```

Looks like it's all good. Let's trace the fixtures for one test file to see if the different scoping worked as expected:

```
$ pytest --setup-show tests/func/test_add.py
======================== test session starts ========================
collected 3 items

tests/func/test_add.py
SETUP S tmpdir_factory
SETUP S tasks_db_session (fixtures used: tmpdir_factory)
SETUP F tasks_db (fixtures used: tasks_db_session)
tests/func/test_add.py::test_add_returns_valid_id
(fixtures used: tasks_db, tasks_db_session, tmpdir_factory).
TEARDOWN F tasks_db
SETUP F tasks_db (fixtures used: tasks_db_session)
tests/func/test_add.py::test_added_task_has_id_set
(fixtures used: tasks_db, tasks_db_session, tmpdir_factory).
TEARDOWN F tasks_db
SETUP F tasks_db (fixtures used: tasks_db_session)
SETUP S tasks_just_a_few
SETUP F db_with_3_tasks (fixtures used: tasks_db, tasks_just_a_few)
```

```
  tests/func/test_add.py::test_add_increases_count
  (fixtures used: db_with_3_tasks, tasks_db, tasks_db_session,
  tasks_just_a_few, tmpdir_factory).
  TEARDOWN F db_with_3_tasks
  TEARDOWN F tasks_db
  TEARDOWN S tasks_just_a_few
  TEARDOWN S tasks_db_session
  TEARDOWN S tmpdir_factory

  ====================== 3 passed in 0.03 seconds ======================
```

Yep. Looks right. tasks_db_session is called once per session, and the quicker tasks_db now just cleans out the database before each test.

Specifying Fixtures with usefixtures

So far, if you wanted a test to use a fixture, you put it in the parameter list. You can also mark a test or a class with @pytest.mark.usefixtures('fixture1', 'fixture2'). usefixtures takes a string that is composed of a comma-separated list of fixtures to use. It doesn't make sense to do this with test functions—it's just more typing. But it does work well for test classes:

ch3/test_scope.py

```
@pytest.mark.usefixtures('class_scope')
class TestSomething():
    """Demo class scope fixtures."""

    def test_3(self):
        """Test using a class scope fixture."""

    def test_4(self):
        """Again, multiple tests are more fun."""
```

Using usefixtures is almost the same as specifying the fixture name in the test method parameter list. The one difference is that the test can use the return value of a fixture only if it's specified in the parameter list. A test using a fixture due to usefixtures cannot use the fixture's return value.

Using autouse for Fixtures That Always Get Used

So far in this chapter, all of the fixtures used by tests were named by the tests (or used usefixtures for that one class example). However, you can use autouse=True to get a fixture to run all of the time. This works well for code you want to run at certain times, but tests don't really depend on any system state or data from the fixture. Here's a rather contrived example:

ch3/test_autouse.py

```
"""Demonstrate autouse fixtures."""

import pytest
import time


@pytest.fixture(autouse=True, scope='session')
def footer_session_scope():
"""Report the time at the end of a session."""
yield
now = time.time()
print('--')
print('finished : {}'.format(time.strftime('%d %b %X', time.localtime(now))))
print('-----------------')


@pytest.fixture(autouse=True)
def footer_function_scope():
"""Report test durations after each function."""
start = time.time()
yield
stop = time.time()
delta = stop - start
print('\ntest duration : {:0.3} seconds'.format(delta))


def test_1():
"""Simulate long-ish running test."""
time.sleep(1)


def test_2():
"""Simulate slightly longer test."""
```

```
    time.sleep(1.23)
```

We want to add test times after each test, and the date and current time at the end of the session. Here's what these look like:

```
$ cd /path/to/code/ch3
$ pytest -v -s test_autouse.py
===================== test session starts ======================
collected 2 items

test_autouse.py::test_1 PASSED
test duration : 1.0 seconds

test_autouse.py::test_2 PASSED
test duration : 1.24 seconds
--
finished : 25 Jul 16:18:27
-----------------
=================== 2 passed in 2.25 seconds ===================
```

The autouse feature is good to have around. But it's more of an exception than a rule. Opt for named fixtures unless you have a really great reason not to.

Now that you've seen autouse in action, you may be wondering why we didn't use it for tasks_db in this chapter. In the Tasks project, I felt it was important to keep the ability to test what happens if we try to use an API function before db initialization. It should raise an appropriate exception. But we can't test this if we force good initialization on every test.

Renaming Fixtures

The name of a fixture, listed in the parameter list of tests and other fixtures using it, is usually the same as the function name of the fixture. However, pytest allows you to rename fixtures with a name parameter to @pytest.fixture():

ch3/test_rename_fixture.py

```
"""Demonstrate fixture renaming."""

import pytest


@pytest.fixture(name='lue')
def ultimate_answer_to_life_the_universe_and_everything():
"""Return ultimate answer."""
return 42


def test_everything(lue):
"""Use the shorter name."""
assert lue == 42
```

Here, lue is now the fixture name, instead of fixture_with_a_name_much_longer_than_lue. That name even shows up if we run it with --setup-show:

```
$ pytest --setup-show test_rename_fixture.py
======================== test session starts ========================
collected 1 items

test_rename_fixture.py
SETUP F lue
test_rename_fixture.py::test_everything_2 (fixtures used: lue).
TEARDOWN F lue

===================== 1 passed in 0.01 seconds ======================
```

If you need to find out where lue is defined, you can add the pytest option --fixtures and give it the filename for the test. It lists all the fixtures available for the test, including ones that have been renamed:

```
$ pytest --fixtures test_rename_fixture.py
======================== test session starts ========================
...
```

---------- fixtures defined from test_rename_fixture ----------
lue
Return ultimate answer.

================= no tests ran in 0.01 seconds =================

Most of the output is omitted—there's a lot there. Luckily, the fixtures we defined are at the bottom, along with where they are defined. We can use this to look up the definition of lue. Let's use that in the Tasks project:

```
$ cd /path/to/code/ch3/b/tasks_proj
$ pytest --fixtures tests/func/test_add.py
======================== test session starts ========================
...
tmpdir_factory
Return a TempdirFactory instance for the test session.
tmpdir
Return a temporary directory path object which is
unique to each test function invocation, created as
a sub directory of the base temporary directory.
The returned object is a `py.path.local`_path object.

------------------ fixtures defined from conftest -------------------
tasks_db_session
Connect to db before tests, disconnect after.
tasks_db
An empty tasks db.
tasks_just_a_few
All summaries and owners are unique.
tasks_mult_per_owner
Several owners with several tasks each.
db_with_3_tasks
Connected db with 3 tasks, all unique.
db_with_multi_per_owner
Connected db with 9 tasks, 3 owners, all with 3 tasks.

=================== no tests ran in 0.01 seconds ====================
```

Cool. All of our conftest.py fixtures are there. And at the bottom of the builtin list is the tmpdir and tmpdir_factory that we used also.

Parametrizing Fixtures

In *Parametrized Testing*, we parametrized tests. We can also parametrize fixtures. We still use our list of tasks, list of task identifiers, and an equivalence function, just as before:

ch3/b/tasks_proj/tests/func/test_add_variety2.py

```
import pytest
import tasks
from tasks import Task

tasks_to_try = (Task('sleep', done=True),
Task('wake', 'brian'),
Task('breathe', 'BRIAN', True),
Task('exercise', 'BrIaN', False))

task_ids = ['Task({},{},{})'.format(t.summary, t.owner, t.done)
for t in tasks_to_try]


def equivalent(t1, t2):
"""Check two tasks for equivalence."""
return ((t1.summary == t2.summary) and
(t1.owner == t2.owner) and
(t1.done == t2.done))
```

But now, instead of parametrizing the test, we will parametrize a fixture called a_task:

ch3/b/tasks_proj/tests/func/test_add_variety2.py

```
@pytest.fixture(params=tasks_to_try)
def a_task(request):
"""Using no ids."""
return request.param


def test_add_a(tasks_db, a_task):
"""Using a_task fixture (no ids)."""
task_id = tasks.add(a_task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, a_task)
```

The request listed in the fixture parameter is another builtin fixture that represents the calling state of the fixture. You'll explore it more in the next chapter. It has a field param that is filled in

with one element from the list assigned to params in @pytest.fixture(params=tasks_to_try).

The a_task fixture is pretty simple—it just returns the request.param as its value to the test using it. Since our task list has four tasks, the fixture will be called four times, and then the test will get called four times:

```
$ cd /path/to/code/ch3/b/tasks_proj/tests/func
$ pytest -v test_add_variety2.py::test_add_a
==================== test session starts ====================
collected 4 items

test_add_variety2.py::test_add_a[a_task0] PASSED
test_add_variety2.py::test_add_a[a_task1] PASSED
test_add_variety2.py::test_add_a[a_task2] PASSED
test_add_variety2.py::test_add_a[a_task3] PASSED

=================== 4 passed in 0.03 seconds ===================
```

We didn't provide ids, so pytest just made up some names by appending a number to the name of the fixture. However, we can use the same string list we used when we parametrized our tests:

ch3/b/tasks_proj/tests/func/test_add_variety2.py

```
@pytest.fixture(params=tasks_to_try, ids=task_ids)
def b_task(request):
"""Using a list of ids."""
return request.param


def test_add_b(tasks_db, b_task):
"""Using b_task fixture, with ids."""
task_id = tasks.add(b_task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, b_task)
```

This gives us better identifiers:

```
$ pytest -v test_add_variety2.py::test_add_b
==================== test session starts ====================
collected 4 items

test_add_variety2.py::test_add_b[Task(sleep,None,True)] PASSED
test_add_variety2.py::test_add_b[Task(wake,brian,False)] PASSED
test_add_variety2.py::test_add_b[Task(breathe,BRIAN,True)] PASSED
test_add_variety2.py::test_add_b[Task(exercise,BrIaN,False)] PASSED
```

```
================== 4 passed in 0.04 seconds ==================
```

We can also set the ids parameter to a function we write that provides the identifiers. Here's what it looks like when we use a function to generate the identifiers:

ch3/b/tasks_proj/tests/func/test_add_variety2.py

```
def id_func(fixture_value):
"""A function for generating ids."""
t = fixture_value
return 'Task({},{},{})'.format(t.summary, t.owner, t.done)


@pytest.fixture(params=tasks_to_try, ids=id_func)
def c_task(request):
"""Using a function (id_func) to generate ids."""
return request.param


def test_add_c(tasks_db, c_task):
"""Use fixture with generated ids."""
task_id = tasks.add(c_task)
t_from_db = tasks.get(task_id)
assert equivalent(t_from_db, c_task)
```

The function will be called from the value of each item from the parametrization. Since the parametrization is a list of Task objects, id_func() will be called with a Task object, which allows us to use the namedtuple accessor methods to access a single Task object to generate the identifier for one Task object at a time. It's a bit cleaner than generating a full list ahead of time, and looks the same:

```
$ pytest -v test_add_variety2.py::test_add_c
==================== test session starts ====================
collected 4 items

test_add_variety2.py::test_add_c[Task(sleep,None,True)] PASSED
test_add_variety2.py::test_add_c[Task(wake,brian,False)] PASSED
test_add_variety2.py::test_add_c[Task(breathe,BRIAN,True)] PASSED
test_add_variety2.py::test_add_c[Task(exercise,BrIaN,False)] PASSED

================== 4 passed in 0.04 seconds ==================
```

With parametrized functions, you get to run that function multiple times. But with parametrized fixtures, every test function that uses that fixture will be called multiple times. Very powerful.

Parametrizing Fixtures in the Tasks Project

Now, let's see how we can use parametrized fixtures in the Tasks project. So far, we used TinyDB for all of the testing. But we want to keep our options open until later in the project. Therefore, any code we write, and any tests we write, should work with both TinyDB and with MongoDB.

The decision (in the code) of which database to use is isolated to the start_tasks_db() call in the tasks_db_session fixture:

ch3/b/tasks_proj/tests/conftest.py

```
import pytest
import tasks
from tasks import Task


@pytest.fixture(scope='session')
def tasks_db_session(tmpdir_factory):
"""Connect to db before tests, disconnect after."""
temp_dir = tmpdir_factory.mktemp('temp')
tasks.start_tasks_db(str(temp_dir), 'tiny')
yield
tasks.stop_tasks_db()


@pytest.fixture()
def tasks_db(tasks_db_session):
"""An empty tasks db."""
tasks.delete_all()
```

The db_type parameter in the call to start_tasks_db() isn't magic. It just ends up switching which subsystem gets to be responsible for the rest of the database interactions:

tasks_proj/src/tasks/api.py

```
def start_tasks_db(db_path, db_type): # type: (str, str) -> None
"""Connect API functions to a db."""
if not isinstance(db_path, string_types):
raise TypeError('db_path must be a string')
global _tasksdb
if db_type == 'tiny':
import tasks.tasksdb_tinydb
_tasksdb = tasks.tasksdb_tinydb.start_tasks_db(db_path)
elif db_type == 'mongo':
import tasks.tasksdb_pymongo
```

```
    _tasksdb = tasks.tasksdb_pymongo.start_tasks_db(db_path)
else:
    raise ValueError("db_type must be a 'tiny' or 'mongo'")
```

To test MongoDB, we need to run all the tests with db_type set to mongo. A small change does the trick:

ch3/c/tasks_proj/tests/conftest.py

```
import pytest
import tasks
from tasks import Task


#@pytest.fixture(scope='session', params=['tiny',])
@pytest.fixture(scope='session', params=['tiny', 'mongo'])
def tasks_db_session(tmpdir_factory, request):
    """Connect to db before tests, disconnect after."""
    temp_dir = tmpdir_factory.mktemp('temp')
    tasks.start_tasks_db(str(temp_dir), request.param)
    yield # this is where the testing happens
    tasks.stop_tasks_db()


@pytest.fixture()
def tasks_db(tasks_db_session):
    """An empty tasks db."""
    tasks.delete_all()
```

Here I added params=['tiny','mongo'] to the fixture decorator. I added request to the parameter list of temp_db, and I set db_type to request.param instead of just picking 'tiny' or 'mongo'.

When you set the --verbose or -v flag with pytest running parametrized tests or parametrized fixtures, pytest labels the different runs based on the value of the parametrization. And because the values are already strings, that works great.

Installing MongoDB



To follow along with MongoDB testing, make sure MongoDB and pymongo are installed. I've been testing with the community edition of MongoDB, found at https://www.mongodb.com/download-center. pymongo is installed with pip—pip install pymongo. However, using MongoDB is not necessary to follow along with the rest of the book; it's used in this example and in a debugger example in Chapter 7.

Here's what we have so far:

```
$ cd /path/to/code/ch3/c/tasks_proj
$ pip install pymongo
$ pytest -v --tb=no
==================== test session starts ====================
collected 92 items

test_add.py::test_add_returns_valid_id[tiny] PASSED
test_add.py::test_added_task_has_id_set[tiny] PASSED
test_add.py::test_add_increases_count[tiny] PASSED
test_add_variety.py::test_add_1[tiny] PASSED
test_add_variety.py::test_add_2[tiny-task0] PASSED
test_add_variety.py::test_add_2[tiny-task1] PASSED
...
test_add.py::test_add_returns_valid_id[mongo] FAILED
test_add.py::test_added_task_has_id_set[mongo] FAILED
test_add.py::test_add_increases_count[mongo] PASSED
test_add_variety.py::test_add_1[mongo] FAILED
test_add_variety.py::test_add_2[mongo-task0] FAILED
...
============= 42 failed, 50 passed in 4.94 seconds =============
```

Hmm. Bummer. Looks like we'll need to do some debugging before we let anyone use the Mongo version. You'll take a look at how to debug this in *pdb: Debugging Test Failures*. Until then, we'll use the TinyDB version.

Exercises

1. Create a test file called test_fixtures.py.
2. Write a few data fixtures—functions with the @pytest.fixture() decorator—that return some data. Perhaps a list, or a dictionary, or a tuple.
3. For each fixture, write at least one test function that uses it.
4. Write two tests that use the same fixture.
5. Run pytest --setup-show test_fixtures.py. Are all the fixtures run before every test?
6. Add scope='module' to the fixture from Exercise 4.
7. Re-run pytest --setup-show test_fixtures.py. What changed?
8. For the fixture from Exercise 6, change return <data> to yield <data>.
9. Add print statements before and after the yield.
10. Run pytest -s -v test_fixtures.py. Does the output make sense?

What’s Next

The pytest fixture implementation is flexible enough to use fixtures like building blocks to build up test setup and teardown, and to swap in and out different chunks of the system (like swapping in Mongo for TinyDB). Because fixtures are so flexible, I use them heavily to push as much of the setup of my tests into fixtures as I can.

In this chapter, you looked at pytest fixtures you write yourself, as well as a couple of builtin fixtures, tmpdir and tmpdir_factory. You’ll take a closer look at the builtin fixtures in the next chapter.

Copyright © 2017, The Pragmatic Bookshelf.